

Java Exceptions Best Practices



Topics

- What is Exception for?
- Best practices in **designing** Exception APIs
- Best practices in **using** Exception APIs

**What is Exception
for?**

What Exception Can Tell you

- What went wrong?
 - > type of exception thrown
- Where did it go wrong?
 - > exception stack trace
- Why did it go wrong?
 - > exception message

Best Practices for Designing the API

Checked Exception or Unchecked Exception?

- Ask yourself, "What action can the client code take when the exception occurs?"
- If the client can take some alternate action to recover from the exception, make it a checked exception
 - > By useful, I mean taking steps to recover from the exception and not just logging the exception.
 - > If the client cannot do anything useful, then make the exception unchecked.

Checked Exception or Unchecked Exception? (Continued)

- In general, prefer unchecked exceptions for all programming errors
 - > Unchecked exceptions have the benefit of not forcing the client API to explicitly deal with them.
 - > Unchecked exceptions propagate to where you want to catch them, or they go all the way out and get reported.
 - > Use built-in unchecked exceptions, such as `NullPointerException`, `IllegalArgumentException`, and `IllegalStateException` whenever possible

Preserve encapsulation

- Never let implementation-specific checked exceptions escalate to the higher layers.
 - > For example, do not propagate *SQLException* from data access code to the business objects layer
 - > Business objects layer do not need to know about *SQLException*
- Use one of the two options
 - > Convert *SQLException* into another meaningful checked exception, if the client code is expected to recuperate from the exception
 - > Convert *SQLException* into an unchecked exception, if the client code cannot do anything about it.

Use Unchecked Exception (instead of Silently Suppressing them)

- Bad code (Silently suppressing them)

```
public void dataAccessCode(){  
    try{  
        ..some code that throws SQLException  
    }catch(SQLException ex){  
        ex.printStackTrace(); // Do nothing  
    }  
}
```

- Good code (Convert them into Unchecked exception)

```
public void dataAccessCode(){  
    try{  
        ..some code that throws SQLException  
    }catch(SQLException ex){  
        throw new RuntimeException(ex);  
    }  
}
```

Create new Checked Exception with useful methods (clients can call)

- Bad code

```
public class DuplicateUsernameException  
    extends Exception {}
```

- Good code

```
public class DuplicateUsernameException  
    extends Exception {  
    // These are the methods clients can call to find more  
    // on the exceptions  
    public DuplicateUsernameException  
        (String username){....}  
    public String requestedUsername(){...}  
    public String[] availableNames(){...}  
}
```

Document exceptions

- Use Javadoc's *@throws* tag to document both checked and unchecked exceptions that your API throws.

Best Practices for Using Exceptions

Don't use exceptions for flow control

- Bad practice - uses exception for flow control

```
public void useExceptionsForFlowControl() {  
    try {  
        while (true) {  
            increaseCount();  
        }  
        // The useExceptionsForFlowControl() uses an infinite loop  
        // to increase the count until the exception is thrown. This  
        // not only makes the code difficult to read, but also makes it  
        // slower. Use exception handling only in exceptional situations.  
    } catch (MaximumCountReachedException ex) {  
    }  
    //Continue execution  
}  
  
public void increaseCount()  
    throws MaximumCountReachedException {  
    if (count >= 5000)  
        throw new MaximumCountReachedException();  
}
```

Always Clean up after yourself

- Good practice - Uses finally block for cleaning up

```
public void dataAccessCode(){  
    Connection conn = null;  
    try{  
        conn = getConnection();  
        ..some code that throws SQLException  
    }catch(SQLException ex){  
        ex.printStackTrace();  
    } finally{  
        DBUtil.closeConnection(conn);  
    }  
}
```

Do not suppress/ignore exceptions

- When a method from an API throws a checked exception, it is trying to tell you that you should take some counter action.
- If the checked exception does not make sense to you, do not hesitate to convert it into an unchecked exception and throw it again, but do not ignore it by catching it with `{}` and then continue as if nothing had happened.

Catch Exception from Specific

```
File prefsFile = new File(prefsFilename);  
try {  
    readPreferences(prefsFile);  
}  
catch (FileNotFoundException e) {  
    // alert the user that the specified file  
    // does not exist  
}  
catch (EOFException e) {  
    // alert the user that the end of the file  
    // was reached  
}  
catch (ObjectStreamException e) {  
    // alert the user that the file is corrupted  
}  
catch (IOException e) {  
    // alert the user that some other I/O  
    // error occurred  
}
```


Throw Early (page 1)

- Bad practice

```
public void readPreferences(String filename)  
    throws IllegalArgumentException {
```

```
    InputStream in = new FileInputStream(filename);
```

```
    // more code
```

```
}
```

- Generates verbose stack trace

```
java.lang.NullPointerException  
    at java.io.FileInputStream.open(Native Method)  
    at java.io.FileInputStream.<init>(FileInputStream.java:103)  
    at jcheckbook.JCheckbook.readPreferences(JCheckbook.java:225)  
    at jcheckbook.JCheckbook.startup(JCheckbook.java:116)  
    at jcheckbook.JCheckbook.<init>(JCheckbook.java:27)  
    at jcheckbook.JCheckbook.main(JCheckbook.java:318)
```

Throw Early (page 2)

- Good practice - throw an exception at early detection

```
public void readPreferences(String filename)
    throws IllegalArgumentException {
    if (filename == null) {
        throw new IllegalArgumentException ("filename is null");
    }

    InputStream in = new FileInputStream(filename);
    // ...
}
```

- Generates closer-to-the-problem-source report

```
java.lang.NullPointerException
    at jcheckbook.JCheckbook.readPreferences(JCheckbook.java:225)
    at jcheckbook.JCheckbook.startup(JCheckbook.java:116)
    at jcheckbook.JCheckbook.<init>(JCheckbook.java:27)
    at jcheckbook.JCheckbook.main(JCheckbook.java:318)
```

Catch Exception at Proper place

- A common mistake of many Java developers, both new and experienced, is to catch an exception before the program can handle it in an appropriate manner.
- The Java compiler reinforces this behavior by insisting that checked exceptions either be caught or declared.
- The natural tendency is to immediately wrap the code in a try block and catch the exception to stop the compile from reporting errors.

Catch Exception at Proper place

// Bad practice

```
public void readPreferences(String filename) {
```

```
    InputStream in = null;
```

```
    // DO NOT DO THIS!!!
```

```
    try{
```

```
        in = new FileInputStream(filename);
```

```
    }
```

```
    catch (FileNotFoundException e){
```

```
        logger.log(e);
```

```
    }
```

```
    in.read(...);
```

```
}
```

Catch Exception at Proper place

```
// Good practice
public void readPreferences(String filename)
    throws IllegalArgumentException,
        FileNotFoundException, IOException{

    if (filename == null){
        throw new IllegalArgumentException ("filename is null");
    } //if

    ...

    InputStream in = new FileInputStream(filename);

    ...
}
```

Log exceptions just once

- Logging the same exception stack trace more than once can confuse the programmer examining the stack trace about the original source of exception.
- Bad practice

```
catch (NoSuchMethodException e) {  
    LOG.debug("Using cache policy A");  
    LOG.debug("Using retry policy B");  
}
```

Log exceptions just once (cont.)

- Logging the same exception stack trace more than once can confuse the programmer examining the stack trace about the original source of exception.
- Either log the exception, or throw it, but never do both.
- Bad practice - could end up logging multiple times for a single problem

```
catch (NoSuchMethodException e) {  
    LOG.error("Blah", e);  
    throw new MyServiceException("Blah", e);  
}
```

Thank You!

